

# StudyMate AI, Product Brief & Frontend Architecture

## 1. Product Overview

**Product Name:** StudyMate AI

**Product Description:** StudyMate AI is an AI-powered study assistant designed to help university students understand academic concepts, organize their learning, and create personalized revision materials.

The product provides an interactive chat experience where students can ask questions and receive explanations from an AI assistant. In addition to conversation, StudyMate AI can perform assistive actions such as summarizing study content, generating flashcards, and creating practice quizzes.

A key part of the product is that AI-generated content is not automatically treated as final. Users can review, edit, approve, or reject generated study materials before saving them. This gives the user control over AI-generated content and reduces the risk of relying on inaccurate or unsuitable information.

The product will be developed as an AI-native frontend application using Next.js and will be designed so that the AI provider can be changed without requiring major changes to the frontend application.

**Product Goal:** The primary goal of StudyMate AI is to make studying more interactive and efficient by combining conversational AI with practical study-assistance tools while keeping the student in control of AI-generated content.

**Core Product Experience:** The core experience will follow this pattern:

**Ask → Generate → Review → Approve/Modify → Save**

This allows users to interact with the AI through conversation while also using AI-powered actions to create useful study resources.

## 2. Target Users

**Primary Users:** StudyMate AI is primarily designed for university students who need support understanding academic concepts, reviewing course materials, and preparing for examinations or assessments.

These users may study independently and need an accessible tool that can explain difficult concepts, transform study material into revision resources, and provide interactive practice.

### User Characteristics

The primary users are expected to:

- Study multiple academic subjects and manage different types of course material.
- Encounter concepts that require additional explanation or examples.
- Need to revise information efficiently before tests and examinations.

- Use digital tools as part of their study process.
- Want to interact with an AI assistant through natural language rather than relying only on traditional search.
- Need control over AI-generated study materials before using or saving them.

**Secondary Users:** A secondary user group may include tutors, teaching assistants, or educators who want to experiment with AI-assisted preparation of educational materials.

However, the initial product scope will prioritize university students. Supporting educators as a dedicated user group can be considered in a future version.

**Jobs-to-be-Done:** The main jobs StudyMate AI is designed to help users accomplish are:

1. Understand: When I encounter a difficult academic concept, I want to ask questions and receive a clear explanation so that I can understand the topic.
2. Summarize: When I have lengthy study material, I want to generate a concise summary so that I can identify and review the most important information.
3. Create Revision Materials: When I am preparing for an assessment, I want to generate flashcards and practice questions so that I can test my understanding.
4. Review AI Output: When the AI generates study material, I want to review and edit it before saving it so that I remain in control of what I use for studying.
5. Continue Learning: When I have follow-up questions, I want the AI to maintain the context of the conversation so that I can continue learning without repeatedly explaining the same topic.

### 3. Core AI Use Cases

StudyMate AI will use AI primarily to support understanding, summarization, revision, and interactive practice. The initial product will focus on a small number of clearly defined AI capabilities so that they can be implemented and evaluated across later development modules.

**3.1 Conversational Study Assistant:** Users can ask the AI questions about academic concepts through a chat interface. The AI should provide explanations that are clear, structured, and appropriate for the user's question. Users should also be able to ask follow-up questions while maintaining the context of the current conversation.

**Example: User:** "Explain photosynthesis in simple terms."

**AI:** Provides an explanation.

**User:** "What role does chlorophyll play?"

**AI:** Answers the follow-up question using the context of the previous conversation.

**3.2 Content Summarization:** Users can provide study content and request a summary. The AI should transform lengthy content into a more concise format while attempting to preserve the key information.

**3.3 Revision Material Generation:** Users can request AI-generated revision materials based on a topic or provided study content. The initial version will support flashcard generation.

**Example:** User provides study material and selects Generate Flashcards.

The AI produces a collection of question-and-answer flashcards that the user can review.

The generated flashcards should enter a review state before they are saved as part of the user's study materials.

**3.4 Practice Quiz Generation:** Users can ask the AI to generate practice questions based on a topic or study material. The initial version will focus on generating a small set of questions that users can use to test their understanding. The system should clearly distinguish AI-generated practice questions from verified examination questions.

**3.5 AI Output Review and Approval:** AI-generated content should not automatically become saved user content. When StudyMate AI generates flashcards, summaries, or quizzes, the user should be able to:

- Review the generated content.
- Edit the content when necessary.
- Approve the content.
- Reject the content.
- Regenerate the content when the result is not useful.

This review step keeps the user in control of AI-generated information and provides an important safeguard against unsuitable or inaccurate output.

**3.6 AI Failure and Fallback Behavior:** The product must account for situations where an AI request cannot be completed.

Possible causes include:

- Network failure.
- AI provider errors.
- Request timeouts.
- Invalid or empty user input.
- Temporary service unavailability.

When an AI request fails, the interface should communicate the problem clearly and provide an appropriate recovery action, such as retrying the request. The system should not present an unsuccessful or incomplete AI response as if it were a successful result.

## 4. User Journeys & Interaction Flows

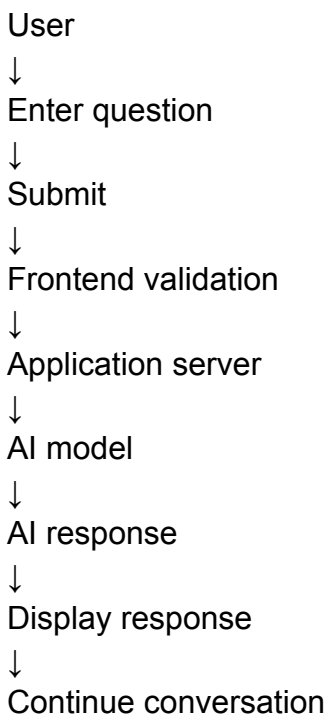
StudyMate AI will be designed around four primary interaction flows: conversational chat, assistive AI actions, review and approval of generated content, and fallback behavior when an AI operation cannot be completed successfully.

**4.1 Chat Flow:** The chat flow allows users to have an ongoing conversation with the AI study assistant.

### Flow:

1. User opens the StudyMate AI chat interface.
2. User enters an academic question or request.
3. User submits the message.
4. The frontend validates that the input is not empty.
5. The message is sent to the application server.
6. The server sends the relevant conversation context and user request to the selected AI provider.
7. The AI generates a response.
8. The response is returned to the frontend.
9. The interface displays the AI response in the conversation.
10. The conversation state is updated so that the user can continue with follow-up questions.

### Flow representation:



**4.2 Assistive Action Flow:** Assistive actions allow users to request a specific transformation or generation task instead of having a general conversation.

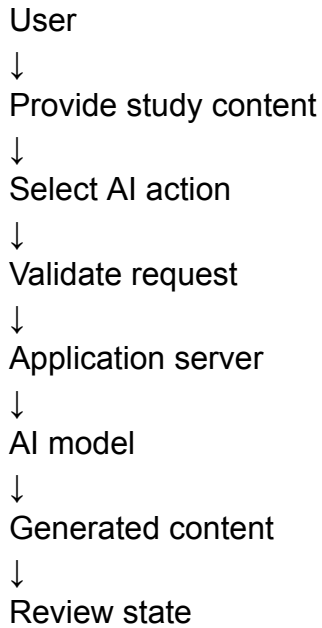
Examples include summarizing content, generating flashcards, and generating practice quizzes.

### Flow:

1. User provides or selects study content.
2. User selects an AI action such as **Summarize**, **Generate Flashcards**, or **Generate Quiz**.
3. The frontend validates the request.
4. The request is sent to the application server.
5. The server sends the appropriate instructions and content to the AI provider.
6. The AI generates the requested result.
7. The generated result is returned to the frontend.

8. The result is displayed in a review state.
9. The user can review the generated content before deciding what to do with it.

### Flow representation:

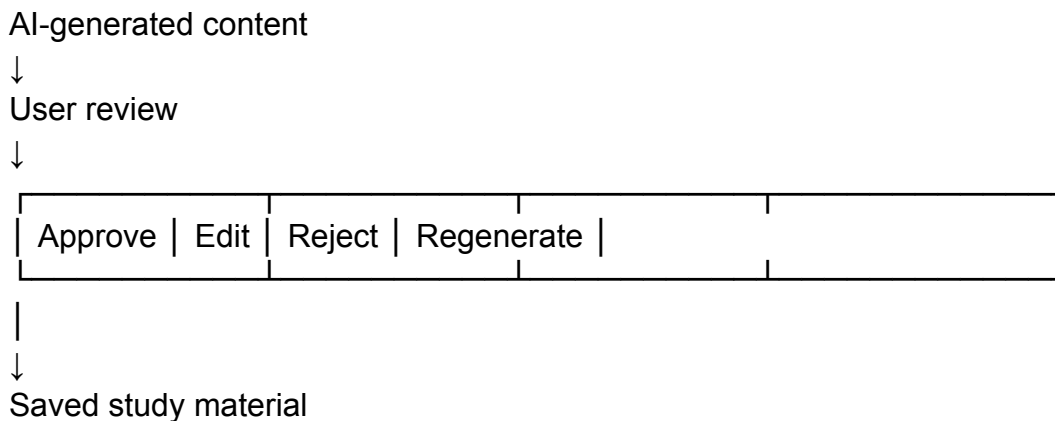


**4.3 Review and Approval Flow:** AI-generated study materials will enter a review state before being saved. This gives the user control over the final content.

### Flow:

1. AI generates content.
2. The generated content is displayed to the user.
3. User reviews the result.
4. User chooses one of the available actions:
  - **Approve:** Save the content.
  - **Edit:** Modify the generated content before saving.
  - **Reject:** Discard the generated content.
  - **Regenerate:** Request a new result from the AI.
5. If the user approves the content, it moves into the saved state.

### Flow representation:



**4.4 Fallback and Error Flow:** The application must provide a clear recovery experience when an AI operation fails.

Potential failure conditions include:

- Network interruption.
- AI provider error.
- Request timeout.
- Invalid request.
- Empty user input.
- Temporary service unavailability.

**Flow:**

1. User submits a request.
2. The system attempts to process the request.
3. If the operation succeeds, the result is displayed normally.
4. If the operation fails, the application displays an understandable error message.
5. The user is provided with an appropriate recovery action, such as **Retry**.
6. The user can retry the request without losing the existing conversation or study context where possible.

**Failure flow representation:**

User request

↓

Processing

↓

Did the request succeed?

**YES** → Display result → Continue

**NO** → Display error → Retry → Process request again

The interface should not silently fail, display a misleading successful state, or leave the user uncertain about whether the request was completed.

**4.5 Overall User Journey:** The main StudyMate AI journey connects the different flows into one continuous experience:

Open StudyMate AI

↓

Start conversation

↓

Ask a question

↓

Receive AI explanation

↓

Request an assistive action

↓  
Generate study material  
↓  
Review generated content  
↓  
Edit / Approve / Reject / Regenerate  
↓  
Save approved material  
↓  
Continue studying through chat or additional AI actions

## 5. System Architecture & Boundaries

StudyMate AI will use a client-server architecture in which the browser handles the user interface while sensitive AI interactions are handled on the server.

### 5.1 Architecture Overview

**User → Next.js Frontend → Application Server → AI Provider → Application Server → Frontend → User**

The frontend will provide the interactive user experience, while the server will manage communication with the AI provider and protect sensitive credentials.

### 5.2 System Boundaries

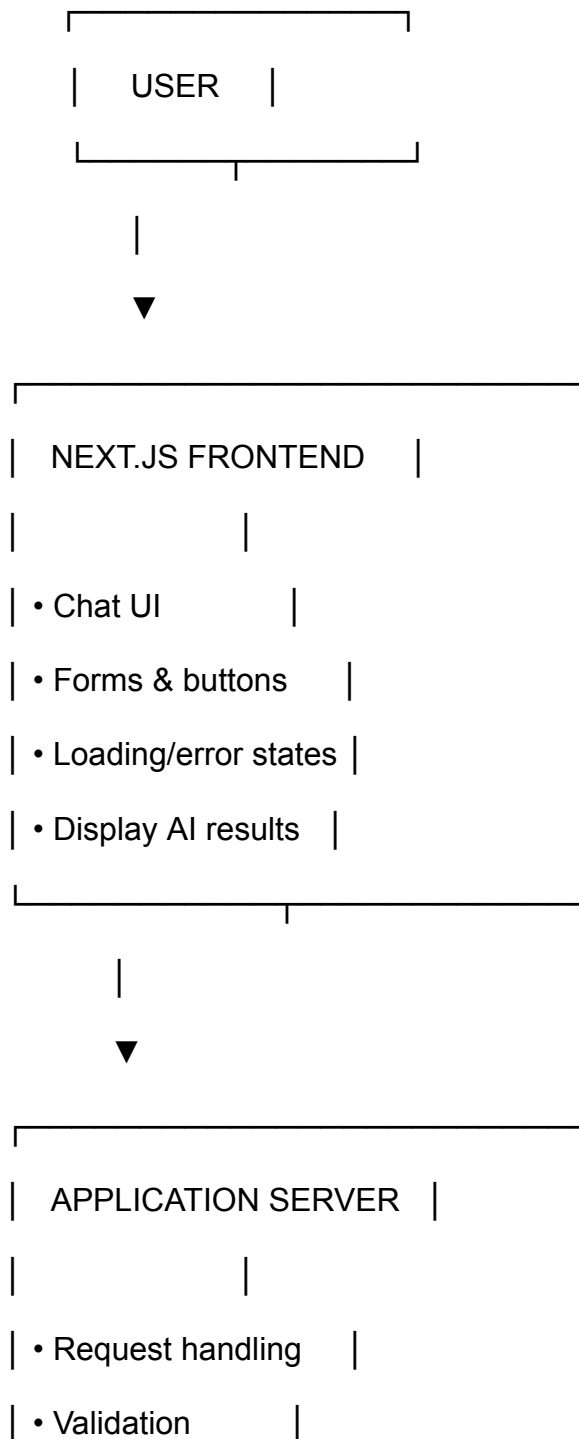
| Layer                     | Responsibilities                                                                                                                      |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>Browser / Frontend</b> | User interface, chat interaction, forms, buttons, loading states, displaying AI responses, and client-side input validation.          |
| <b>Application Server</b> | Process requests, validate requests, manage AI provider communication, protect API keys, and control access to server-side resources. |
| <b>AI Model</b>           | Generate explanations, summaries, flashcards, and practice questions based on the user's request and permitted context.               |
| <b>Data Layer</b>         | Store approved study materials, conversation/session information, and selected user preferences when persistence is introduced.       |

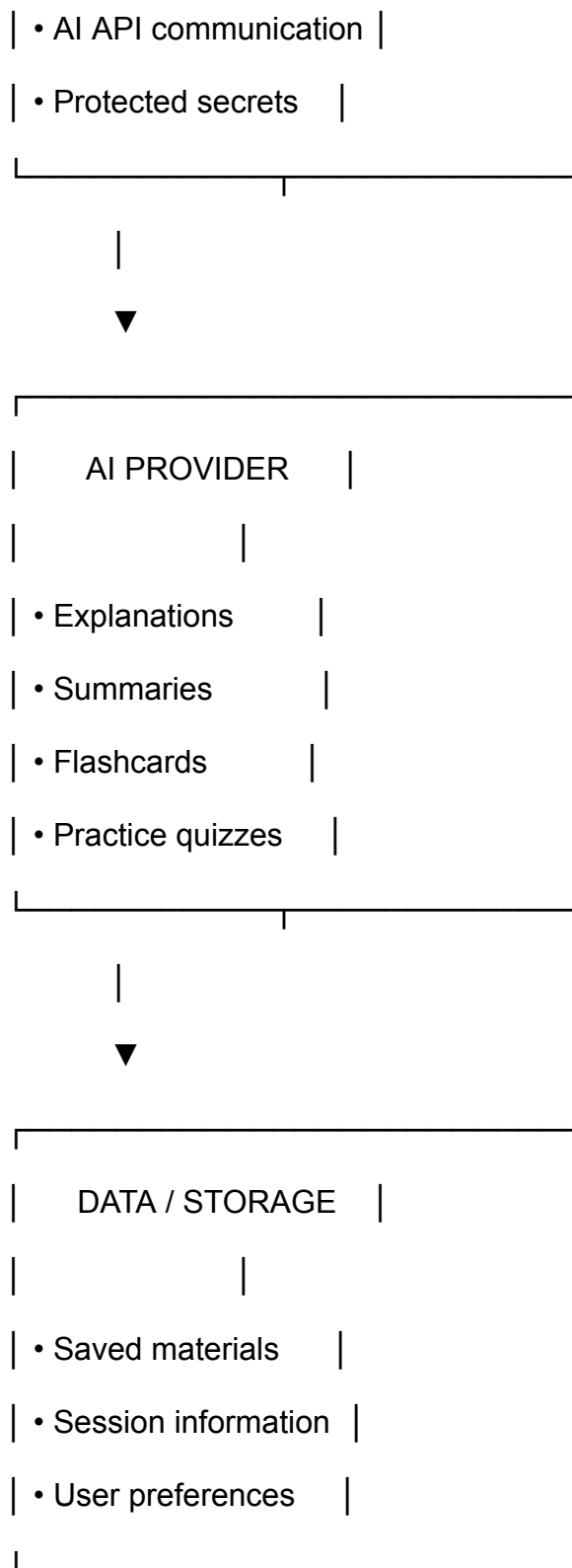
## External Services

AI provider and deployment infrastructure such as Vercel.

**5.3 Security Boundary:** Sensitive credentials must remain on the server and must never be exposed directly to the browser. AI provider API keys, database credentials, and other private environment variables will be stored using server-side environment variables. The frontend will communicate with the application's server rather than directly exposing private AI provider credentials.

## 5.4 Architecture Diagram





Security boundary:

API keys and private credentials remain server-side.

**5.5 AI Provider Strategy:** The initial implementation will use one selected AI provider, while keeping the AI integration replaceable. The frontend should not be tightly coupled to a specific model provider. The Vercel AI SDK will be considered as the abstraction layer for AI interactions so that the underlying provider can be changed with minimal changes to the frontend application.

**5.6 Deployment Boundary:** The Next.js application will be connected to GitHub and deployed through Vercel. Future code changes pushed to the repository should trigger an automatic deployment, allowing the live application to remain synchronized with the project's main branch.

**6. Acceptance Criteria & State Management**

**6.1 Acceptance Criteria**

| Area          | Acceptance Criteria                                                                                                                                  |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Accuracy      | AI-generated responses should directly address the user's request and the interface should clearly identify generated content as AI-generated.       |
| Latency       | The interface should provide a visible loading or streaming state while waiting for an AI response so the user knows the request is being processed. |
| Accessibility | Primary controls, forms, buttons, and interactive elements should be keyboard accessible and have clear labels.                                      |
| Safety        | API keys and private credentials must never be exposed to the browser. AI-generated study materials must enter a review state before being saved.    |
| UX Quality    | Users should always receive clear feedback for loading, success, and error states, and should have an obvious way to recover from failed requests.   |

**6.2 State Management Decisions:** StudyMate AI will need to manage several categories of application state.

**Conversation history:** The current conversation will maintain the messages required to provide context for follow-up questions.

**Session state:** Temporary information such as the current chat session, active AI action, loading state, error state, and content currently being reviewed will be managed during the user's session.

**Saved preferences:** Future versions may allow users to save preferences such as preferred explanation style, study settings, or other personalization options.

**Generated content state:** AI-generated materials will move through defined states such as:

**Generating → Review → Approved / Edited / Rejected:** This state model ensures that generated content is not automatically treated as final user content.

## 8. Technology & AI Provider Decision

**Frontend: Next.js with the App Router** will be used as the primary frontend framework.

**AI Integration:** The initial AI provider for StudyMate AI will be **OpenAI**. The application will be designed so that the AI provider can be replaced in the future without requiring major changes to the frontend architecture.

The **Vercel AI SDK** will be considered as the abstraction layer between the application and the AI provider. This will reduce direct coupling between the frontend application and a specific model provider.

**Deployment:** The application will be hosted on **Vercel** and connected to the project's GitHub repository for automatic deployments.

### Technology Summary

- **Framework:** Next.js
- **Frontend:** React / Next.js App Router
- **AI integration:** Vercel AI SDK
- **Initial AI provider:** OpenAI
- **Version control:** GitHub
- **Deployment:** Vercel
- **Future data storage:** To be selected when persistent storage is introduced